

- Use $\text{\LaTeX}$ to write-up your solutions (in the solution blocks of the source $\text{\LaTeX}$ file of this assignment), and submit the resulting single pdf file at GradeScope by the due date. (Note: As always, **no late submissions** will be allowed, other than one-day late submission with 10% penalty! Within GradeScope, indicate the page number where your solution to each question starts, else we won't be able to grade it!).
- Collaboration is encouraged, but all write-ups must be done individually and independently, and mention your collaborator(s) if any. Same rules apply for codes written for any programming assignments (i.e., write your own code; we will run plagiarism checks on codes).
- If you have referred a book or any other online material for obtaining a solution, please cite the source. Again don't copy the source *as is* - you may use the source to understand the solution, but write-up the solution in your own words.

1. (10 points) [IS HMM BP IN DISGUISE? Hmmm] Consider the following Hidden Markov Model (HMM): The hidden r.v.s are $Y_0, \dots, Y_n$ taking values in $\{1, \dots, a\}$ and the observed r.v.s are $X_0, \dots, X_n$ taking values in $\{1, \dots, b\}$. The parameters are given by the transition probability matrix $T \in \mathbb{R}_+^{a \times a}$, the emission probability matrix $V \in \mathbb{R}_+^{b \times a}$ and the initial probability vector $\pi \in \mathbb{R}_+^a$. Assume all parameters are known to you. Concretely,

$$T_{j,i} = P(Y_{t+1} = j \mid Y_t = i)$$

$$V_{k,i} = P(X_t = k \mid Y_t = i)$$

$$\pi_i = P(Y_0 = i).$$

The joint probability is given as:

$$P(X, Y) = P(Y_0)P(X_0 \mid Y_0) \prod_{t=1}^n P(Y_t \mid Y_{t-1})P(X_t \mid Y_t)$$

Consider the BP algo. from class with a two-phase message-passing schedule rooted at node Y_n .

- (a) (3 points) [FORWARD ALGO. FOR HMM LIKELIHOOD] Derive the evidence probability $P(X = x)$ using the BP algorithm's first phase (bottom-up or forward phase). Simplify these first-phase BP messages by expressing them as recursive updates of

$$\alpha_t^i := P(X_0, X_1, \dots, X_t, Y_t = i) = \sum_{Y_0, \dots, Y_{t-1}} P(X_0, \dots, X_t, Y_0, \dots, Y_{t-1}, Y_t = i). \text{ What are the source}$$

and target nodes of the α_t^i message, and what message is sent from the observed X_t to the hidden Y_t ?

Solution: Consider the set of potentials in the HMM,

$$\phi(Y_1) = P(Y_1) \quad (1a)$$

$$\phi(Y_t) = 1 \quad (1b)$$

$$\phi(Y_{t-1}, Y_t) = P(Y_t | Y_{t-1}) \quad (1c)$$

Consider the set of messages,

$$m_{X_1 \rightarrow Y_1}(Y_1) = \sum_{X_1} P(X_1 | Y_1) \quad (2a)$$

$$m_{Y_1 \rightarrow Y_2}(Y_2) = \sum_{Y_1} \phi(Y_1) \phi(Y_1, Y_2) m_{X_1 \rightarrow Y_1}(Y_1) = \sum_{Y_1} P(Y_1) P(Y_2 | Y_1) P(X_1 | Y_1) \quad (2b)$$

$$\begin{aligned} m_{Y_{t-1} \rightarrow Y_t}(Y_t) &= \sum_{Y_{t-1}} \phi(Y_{t-1}) \phi(Y_{t-1}, Y_t) m_{X_{t-1} \rightarrow Y_{t-1}}(Y_{t-1}) m_{Y_{t-2} \rightarrow Y_{t-1}}(Y_{t-1}) \\ &= \sum_{Y_{t-1}} P(Y_t | Y_{t-1}) P(X_{t-1} | Y_{t-1}) m_{Y_{t-2} \rightarrow Y_{t-1}}(Y_{t-1}) \end{aligned} \quad (2c)$$

Then,

$$m_{Y_1 \rightarrow Y_2}(Y_2) = \sum_{Y_1} P(Y_1) P(Y_2 | Y_1) P(X_1 | Y_1) = \sum_{Y_1} P(X_1, Y_1, Y_2) = P(X_1, Y_2)$$

And,

$$m_{Y_{t-1} \rightarrow Y_t}(Y_t) = \sum_{Y_{t-1}} P(X_1, X_2, \dots, X_{t-1}, Y_{t-1}, Y_t) = P(X_1, X_2, \dots, X_{t-1}, Y_t)$$

$$\text{Then, } P(X_{1:t}) = \sum_{Y_t} P(X_1, X_2, \dots, X_{t-1}, Y_t) = \sum_{Y_t} m_{Y_{t-1} \rightarrow Y_t}(Y_t)$$

Consider,

$$\alpha_t = P(X_1, X_2, \dots, X_{t-1}, X_t, Y_t) = \sum_{Y_{t-1}} P(X_1, X_2, \dots, X_{t-1}, X_t, Y_{t-1}, Y_t)$$

$$= \sum_{Y_{t-1}} P(X_1, X_2, \dots, X_{t-1}, Y_{t-1}) P(Y_t | Y_{t-1}) P(X_t | Y_t)$$

$$= \sum_{Y_{t-1}} \alpha_{t-1} P(Y_t | Y_{t-1}) P(X_t | Y_t)$$

And,

$$m_{Y_t \rightarrow Y_{t+1}}(Y_{t+1}) = P(X_1, X_2, \dots, X_t, Y_{t+1}) = \sum_{Y_t} P(X_1, X_2, \dots, X_t, Y_t, Y_{t+1})$$

$$= \sum_{Y_t} P(Y_{t+1} | Y_t) P(X_1, X_2, \dots, X_t, Y_t) = \sum_{Y_t} P(Y_{t+1} | Y_t) \alpha_t$$

Source Node of $\alpha_t = Y_t$,

Target Node of $\alpha_t = Y_{t+1}$

- (b) (2 points) [VITERBI ALGO. FOR HMM DECODING] What minimal changes will you need to make to the above algorithm so that it computes $\max_y P(X, Y = y)$ instead of $\sum_y P(X, Y = y)$. In particular, how will you change the definition of α_t^i and the corresponding recursive updates? Don't forget to mention also how to initialize/terminate the recursive updates.

Solution: To change the summation to a maximum, we change the definition of α_t^i as follows:

$$\alpha_t = \arg \max_{Y_1, \dots, Y_{t-1}} P(X_1, X_2, \dots, X_t, Y_1, Y_2, \dots, Y_t)$$

$$= \arg \max_{Y_1, \dots, Y_{t-1}} P(X_t | Y_t) P(Y_t | Y_{t-1}) P(X_1, X_2, \dots, X_{t-1}, Y_1, Y_2, \dots, Y_{t-1})$$

$$= \arg \max_{Y_{t-1}} P(X_t | Y_t) P(Y_t | Y_{t-1}) \arg \max_{Y_1, \dots, Y_{t-2}} P(X_1, X_2, \dots, X_{t-1}, Y_1, Y_2, \dots, Y_{t-1})$$

$$= \arg \max_{Y_{t-1}} P(X_t | Y_t) P(Y_t | Y_{t-1}) \alpha_{t-1}$$

$$\alpha_t^i = \arg \max_{Y_{t-1}} P(X_t | Y_t = i) P(Y_t = i | Y_{t-1}) \alpha_{t-1}$$

And,

$$\alpha_0 = \arg \max_{Y_0} P(X_1, Y_1) = P(X_1, Y_1)$$

- (c) (4 points) [BACKWARD ALGO. FOR HMM LEARNING] Simplify your second (top-down or backward) phase BP messages by expressing them as recursive updates of $\beta_t^i := P(X_{t+1}, X_{t+2}, \dots, X_n | Y_t = i)$. What are the source and target nodes of the β_t^i message? How will you use these backward messages along with the forward messages to compute $P(Y_t = i | X)$ (a quantity useful for learning HMM parameters from data)?

Solution: Backward phase BP messages,

$$m_{Y_{t+1} \rightarrow Y_t}(Y_t) = \sum_{Y_{t+1}} \phi(Y_{t+1}) \phi(Y_t, Y_{t+1}) m_{Y_{t+2} \rightarrow Y_{t+1}}(Y_{t+1}) m_{X_{t+1} \rightarrow Y_{t+1}}(Y_{t+1}) \quad (3a)$$

$$= \sum_{Y_{t+1}} P(Y_{t+1} | Y_t) P(X_{t+1} | Y_{t+1}) m_{X_{t+1} \rightarrow Y_{t+1}}(Y_{t+1})$$

And,

$$\begin{aligned} m_{Y_n \rightarrow Y_{n-1}}(Y_{n-1}) &= \sum_{Y_n} P(Y_n | Y_{n-1}) P(X_n | Y_n) \\ &= \sum_{Y_n} P(Y_n | Y_{n-1}) P(X_n | Y_n, Y_{n-1}) = \sum_{Y_n} P(Y_n, X_n | Y_{n-1}) = \sum_{Y_n} P(X_n | Y_{n-1}) \end{aligned}$$

By induction, we can prove,

$$m_{Y_{t+1} \rightarrow Y_t}(Y_t) = P(X_n, \dots, X_{t+1} | Y_t) \quad (3b)$$

Now,

$$m_{Y_{t+1} \rightarrow Y_t}(y_t) = P(X_n, \dots, X_{t+1} | Y_t) \quad (3c)$$

$$\begin{aligned} \beta_t &:= P(X_{t+1}, X_{t+2}, \dots, X_n | Y_t) = m_{Y_{t+1} \rightarrow Y_t}(Y_t) \\ &= \sum_{Y_{t+1}} P(X_{t+1}, X_{t+2}, \dots, X_n, Y_{t+1} | Y_t) \\ &= \sum_{Y_{t+1}} P(X_{t+1} | Y_{t+1}) P(Y_{t+1} | Y_t) P(X_{t+2}, X_{t+3}, \dots, X_n | Y_{t+1}) \\ &= \sum_{Y_{t+1}} P(X_{t+1} | Y_{t+1}) \beta_{t+1} \end{aligned} \quad (3d)$$

Source Node of $\beta_t = Y_{t+1}$,

Target Node of $\beta_t = Y_t$

- (d) (1 point) How many JTs (junction trees) are possible for the HMM model above, and which of these JTs would you choose to get a JT algorithm whose messages are identical to the BP algorithm messages above?

Solution: The HMM model is itself a tree, and hence is trivially a JT. This is the JT whose messages are identical to the ones above.

The number of JTs possible are of the order of $n!$.

2. (10 points) [JT IN THEORY] Let JT be any junction tree of a (chordal or non-chordal) MN H_Φ associated with a set of factors $\phi \in \Phi$ defined over n random variables (with mega-factors grouping these original factors denoted by $\psi_i(C_i)$).

- (a) (3 points) Prove that the number of nodes in the JT is at most n . (Hint: For a given PEO of a chordal graph, assign each maximal clique to the first eliminated vertex in the clique; and count to show that the number of maximal cliques in the chordal graph is at most n .)

Solution: To build a junction tree over an MN H_Φ we first induce an elimination graph on the MN, and then build a JT over maximal cliques of the triangulated graph.

Consider a Perfect Elimination Ordering $X_1, X_2, \dots, X_n$ over the triangularized graph H'_Φ . Since no new edges are added during a perfect elimination, the PEO maintains all original cliques and no new cliques are added.

For $X_i \in \text{PEO}$, the post-neighbours of X_i form a maximal clique. It is maximal, since any other nodes in such a cluster not in the clique would require additional edges on eliminating X_i , which contradicts the fact that this is a PEO. It is also a unique maximal clique since if the post-neighbours formed two separate cliques, then, again, eliminating X_i would introduce edges between the two sets of cliques which would contradict PEO.

Thus, for every X_i in the PEO, there exists only 1 maximal clique. Thus, the number of maximal cliques, i.e. the number of nodes in the JT of H_Φ is thus at most n , which is the number of nodes in the PEO.

- (b) (3 points) For any edge (C_i, C_j) in JT, let sepset $S_{ij} := C_i \cap C_j$, and let $X_{<(i,j)}$ be the set of all variables in the scope of clusters in the C_i side of the tree, and $X_{<(j,i)}$ be the set of all variables in the scope of clusters in the C_j side of the tree. Use separation criteria in H_Φ to prove that $(X_{<(i,j)} \perp X_{<(j,i)} \mid S_{ij})$ in distribution $P_\Phi(\mathcal{X})$.

Solution: Consider an edge (C_i, C_j) in JT with sepset $S_{ij} := C_i \cap C_j$. We show the proof by contradiction.

Suppose $X_{<(i,j)}$ is not separated from $X_{<(j,i)}$ given S_{ij} . Then, there exists a trail from $X \in X_{<(i,j)} \setminus S_{ij}$ to $X' \in X_{<(j,i)} \setminus S_{ij}$.

Then, the JT may have a cluster C_k whose scope contains both X and X' on the C_j side of the tree. However, $X \notin S_{ij}$, which breaks the running intersection property. Similarly, if the JT has a cluster C_k whose scope contains both X and X' on the C_i , $X' \notin S_{ij}$ which again breaks the running intersection property.

Thus, there cannot be a trail from any node in $X_{<(i,j)}$ to any node in $X_{<(j,i)}$ which is not through S_{ij} . Thus, $(X_{<(i,j)} \perp X_{<(j,i)} \mid S_{ij})$.

- (c) (2 points) At convergence (i.e., at the end of two phases of the message-passing schedule; aka (global) calibration) of the JT algorithm, prove that:

$$\mu_{ij}(S_{ij}) = m_{j \rightarrow i}(S_{ij}) m_{i \rightarrow j}(S_{ij})$$

where $\mu_{ij}(S_{ij}) := \sum_{C_i - S_{ij}} \beta_i(C_i)$ and β s are the unnormalized marginals as defined in class. Also, briefly note why $\mu_{ij}(S_{ij})$ is also equal to $\sum_{C_j - S_{ij}} \beta_j(C_j)$.

Solution:

$$\mu_{ij}(S_{ij}) := \sum_{C_i - S_{ij}} \beta_i(C_i) = \sum_{C_i - S_{ij}} \psi_{C_i}(C_i) \prod_{k \in \text{Nbrs}(i)} m_{k \rightarrow i} \quad (4a)$$

$$m_{i \rightarrow j}(S_{ij}) = \sum_{C_i - S_{ij}} \psi_{C_i}(C_i) \prod_{\text{Nbrs}(i) \setminus j} m_{k \rightarrow i}$$

$$m_{j \rightarrow i}(S_{ji}) = \sum_{C_j - S_{ji}} \psi_{C_j}(C_j) \prod_{\text{Nbrs}(j) \setminus i} m_{k \rightarrow j}$$

$$m_{i \rightarrow j}(S_{ij}) m_{j \rightarrow i}(S_{ji}) = \sum_{C_i - S_{ij}} \sum_{C_j - S_{ji}} \psi_{C_i}(C_i) \psi_{C_j}(C_j) \prod_{\text{Nbrs}(i) \setminus j} m_{k \rightarrow i} \prod_{\text{Nbrs}(j) \setminus i} m_{k \rightarrow j}$$

$$= \sum_{C_j - S_{ji}} \psi_{C_i}(C_i) \prod_{\text{Nbrs}(i) \setminus j} m_{k \rightarrow i} \left(\sum_{C_j - S_{ji}} \psi_{C_j}(C_j) \prod_{\text{Nbrs}(j) \setminus i} m_{k \rightarrow j} \right)$$

$$= \sum_{C_j - S_{ji}} \psi_{C_i}(C_i) \prod_{\text{Nbrs}(i) \setminus j} m_{k \rightarrow i} m_{j \rightarrow i}$$

$$= \sum_{C_j - S_{ji}} \psi_{C_i}(C_i) \prod_{\text{Nbrs}(i)} m_{k \rightarrow i} \quad (4b)$$

$$= \mu_{ij}(S_{ij})$$

(From Equations 4a and 4b)

- (d) (2 points) Use above result to prove that a converged JT = (V_{JT}, E_{JT}) can be used to reformulate the joint distribution as:

$$\tilde{P}_{\Phi}(\mathcal{X}) = \frac{\prod_{i \in V_{JT}} \beta_i(C_i)}{\prod_{(ij) \in E_{JT}} \mu_{ij}(S_{ij})}.$$

Solution:
$$\frac{\prod_{i \in V_{JT}} \beta_i(C_i)}{\prod_{(ij) \in E_{JT}} \mu_{ij}(S_{ij})} = \frac{\prod_{i \in V_{JT}} \psi_{C_i}(C_i) \prod_{k \in \text{Nbrs}(i)} m_{k \rightarrow i}}{\prod_{(ij) \in E_{JT}} m_{i \rightarrow j}(S_{ij}) m_{j \rightarrow i}(S_{ji})}$$

$$\therefore \frac{\prod_{i \in V_{JT}} \beta_i(C_i)}{\prod_{(ij) \in E_{JT}} \mu_{ij}(S_{ij})} = \prod_{i \in V_{JT}} \psi_{C_i}(C_i) \left(\frac{\prod_{i \in V_{JT}} \prod_{k \in \text{Nbrs}(i)} m_{k \rightarrow i}}{\prod_{(ij) \in E_{JT}} m_{i \rightarrow j}(S_{ij}) m_{j \rightarrow i}(S_{ji})} \right)$$

Now, the product in the denominator of the bracket multiplies bidirectional messages over each edge. This is equivalent to a product over all vertices of the messages passed from each vertex to the other. This is exactly the numerator in the bracket.

$$\therefore \frac{\prod_{i \in V_{JT}} \beta_i(C_i)}{\prod_{(ij) \in E_{JT}} \mu_{ij}(S_{ij})} = \prod_{i \in V_{JT}} \psi_{C_i}(C_i) = \prod_{C \in JT} \phi_{C_i} C_i$$

$$= \tilde{P}_\Phi(X)$$

3. (10 points) [TOYING AROUND WITH MCMC::MH] Use normal distribution centered at the current state of the chain as a proposal distribution in the Metropolis-Hastings algorithm to sample from the $\text{Gamma}(\theta, 1)$ distribution when θ is a non-integer that is at least 2.
- (a) (3 points) Write down the acceptance probability (after all simplifications). What properties do you need to verify to confirm your method reaches the right stationary distribution after running for a sufficiently long time? Show your verification.

Solution: The acceptance probability $\alpha(x' | x) = \min\{1, \frac{P(x')Q(x | x')}{P(x)Q(x' | x)}\}$

Since, the proposal distribution is Gaussian, it is symmetric, and $Q(x | x') = Q(x' | x)$.

Thus, $\alpha(x' | x) = \min\{1, \frac{P(x')}{P(x)}\}$.

Assuming the chain is ergodic, to confirm that we reach the right stationary distribution, we need to satisfy the detailed balance condition. This asserts that, starting from a random state according to the target distribution π , if we take a random transition to another state according to the transition model, $P(x_{i-1} = x', x_i = x) = P(x_{i-1} = x, x_i = x')$.

Consider two points x_a, x_b drawn from $\pi(x)$ such that $\pi(x_b) \geq \pi(x_a)$.

Then, $P(x_{i-1} = x_a, P(x_i = x_b)) = \pi(x_a)Q(x_b | x_a)\min\{1, \frac{\pi(x_b)Q(x_a | x_b)}{\pi(x_a)Q(x_b | x_a)}\}$

Since the proposal distribution is symmetric,

$$P(x_{i-1} = x_a, P(x_i = x_b)) = \pi(x_a)Q(x_b | x_a) \min\{1, \frac{\pi(x_b)}{\pi(x_a)}\}$$

Since $\pi(x_b) \geq \pi(x_a)$,

$$P(x_{i-1} = x_a, P(x_i = x_b)) = \pi(x_a)Q(x_b | x_a) \times 1$$

Similarly,

$$P(x_{i-1} = x_b, P(x_i = x_a)) = \pi(x_b)Q(x_a | x_b) \min\{1, \frac{\pi(x_a)Q(x_b | x_a)}{\pi(x_b)Q(x_a | x_b)}\}$$

$$= \pi(x_b)Q(x_b | x_a) \frac{\pi(x_a)Q(x_b | x_a)}{\pi(x_b)Q(x_a | x_b)}$$

$$= \pi(x_b)Q(x_b | x_a) \frac{\pi(x_a)}{\pi(x_b)}$$

$$= \pi(x_a)Q(x_b | x_a)$$

$$= \pi(x_a)Q(x_a | x_b) \text{ (Since } Q \text{ is symmetric)}$$

Thus, $P(x_{i-1} = x_a, P(x_i = x_b)) = P(x_{i-1} = x_b, P(x_i = x_a))$.

Thus, the joint distribution of two consecutive samples is symmetric. Therefore, x_i and x_{i-1} have the same marginal distributions and $\pi(x)$ is the stationary distribution of the Markov chain.

- (b) (4 points) Provide your code that simulates 1000 values from $\text{Gamma}(5.5, 1)$, and show the trace plots, and the histogram of X_n (with the gamma density overlaid).

Solution:

```
import numpy as np
import pandas as pd
from scipy.stats import gamma
import matplotlib.pyplot as plt
```

```
class MCMC():
```

```
    def __init__(self, iterations, joint):
        self.x_initial = 1
```



```

self.iterations = iterations
self.joint = joint

def MH(self):
    plt.figure()
    x_plot = np.linspace(0, 30, 100)
    plt.plot(x_plot, self.joint.pdf(x_plot),
             label = 'Gamma_Density')

    x_list = [self.x_initial]

    for i in range(self.iterations):

        x_new = self.x_initial + np.random.normal(0, 3)

        alpha = min(1,
                    self.joint.pdf(x_new)/self.joint.pdf(self.x_
                    initial))

        if np.random.uniform(0, 1) < alpha:

            self.x_initial = x_new

        if i > 1000:

            x_list.append(self.x_initial)

    plt.hist(x_list, bins = 'auto', density=True,
             label='Sampled_Distribution')
    plt.plot(x_list)
    plt.legend()
    plt.show()

if __name__ == '__main__':

    M = MCMC(2000, gamma(a=1, scale=5.5))

    M.MH()

```

Figure 1 and 3 show the histogram plots for X_n for 1000 and 10000 simulations. Figure 2

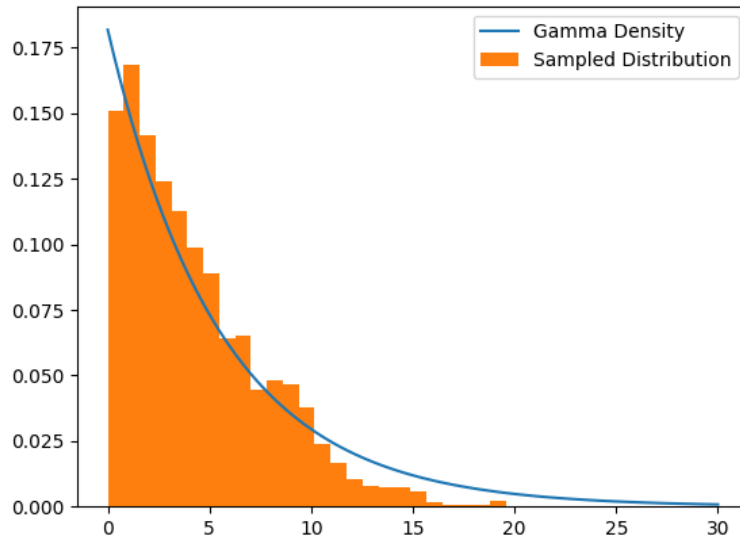


Figure 1: Metropolis Hastings Algorithm: Histogram for 1000 iterations, $\sigma = 1$

and 4 show the corresponding trace plots. Figure 5 shows an example run of the algorithm for other values of gamma parameters.

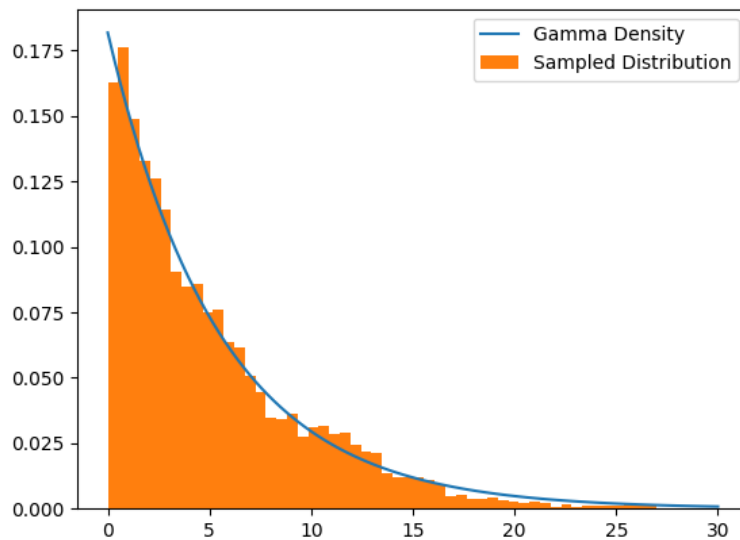


Figure 2: Metropolis Hastings Algorithm: Histogram for 10000 iterations, $\sigma = 1$

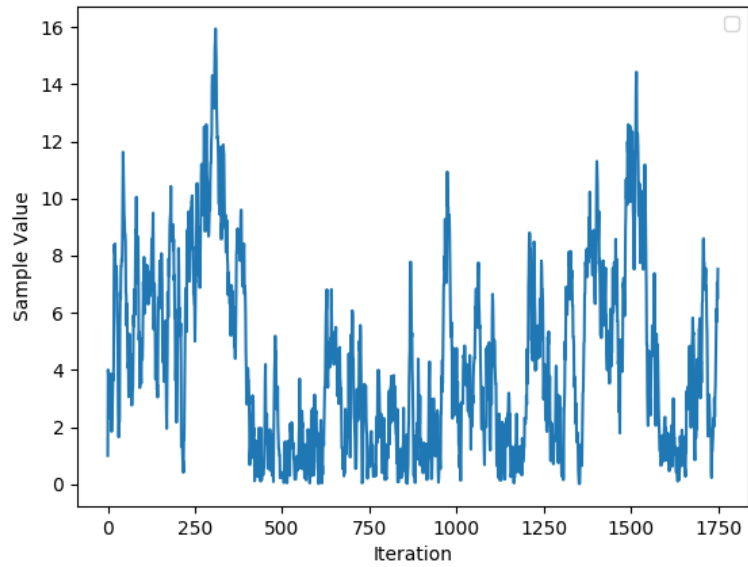


Figure 3: Metropolis Hastings Algorithm: Trace Plot for 1000 iterations, $\sigma = 1$

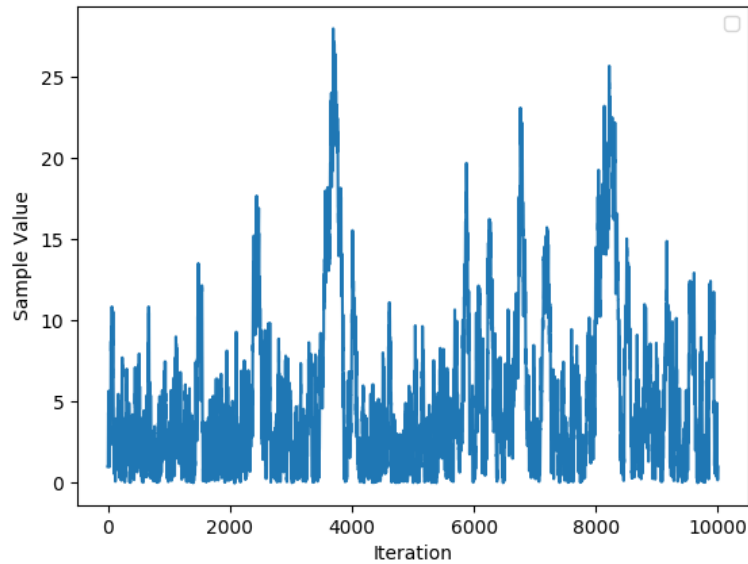


Figure 4: Metropolis Hastings Algorithm: Trace Plot for 10000 iterations, $\sigma = 1$

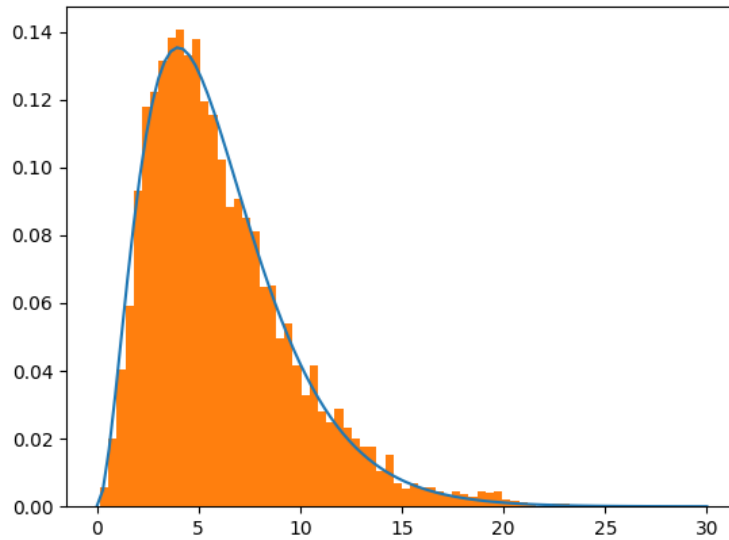


Figure 5: Metropolis Hastings Algorithm: Example Plot for $a = 3$, $\text{scale} = 2$

- (c) (3 points) How did you choose your burn-in time? Report it along with your acceptance rate during the burn-in vs. sample collection periods. How did these values change as a function of the variance parameter of your normal distribution, and what do you think is the optimal variance for fast convergence?

Solution: The burn-in time matters especially when you initialize the Metropolis Hastings algorithm at a point far away from the mean of the function you want to sample. In such a case, it may take a few steps for the chain to "converge". For example, if you start very far away from the mean of your target distribution, then you may take a finite number of steps to reach close to the mean before you can start taking meaningful samples.

Figure 6 shows such a case. Here, a burn-in of approximately 750 steps would be appropriate. The acceptance rate during the burn-in period in this case is higher than that after burn-in.

The dependence of the burn-in period on the variance of the normal distribution is shown in Figure 7. As variance increases, the time taken for the chain to converge reduces. This happens because larger and larger steps can be taken as the variance of the proposal distribution increases. Thus, the burn-in period can also be reduced correspondingly.

The optimal variance for fast convergence is one that is high enough for the chain to con-

verge quickly, but not too large such that the entire space cannot be sampled effectively. Figure 8 shows such a case - the chain converges quickly but the step size is too large to take fine-grained samples.

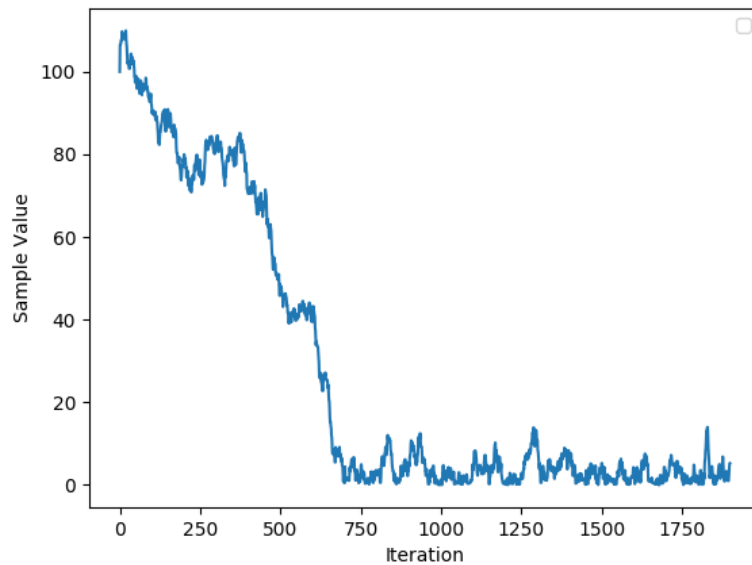


Figure 6: Convergence of the chain occurs after around 750 iterations

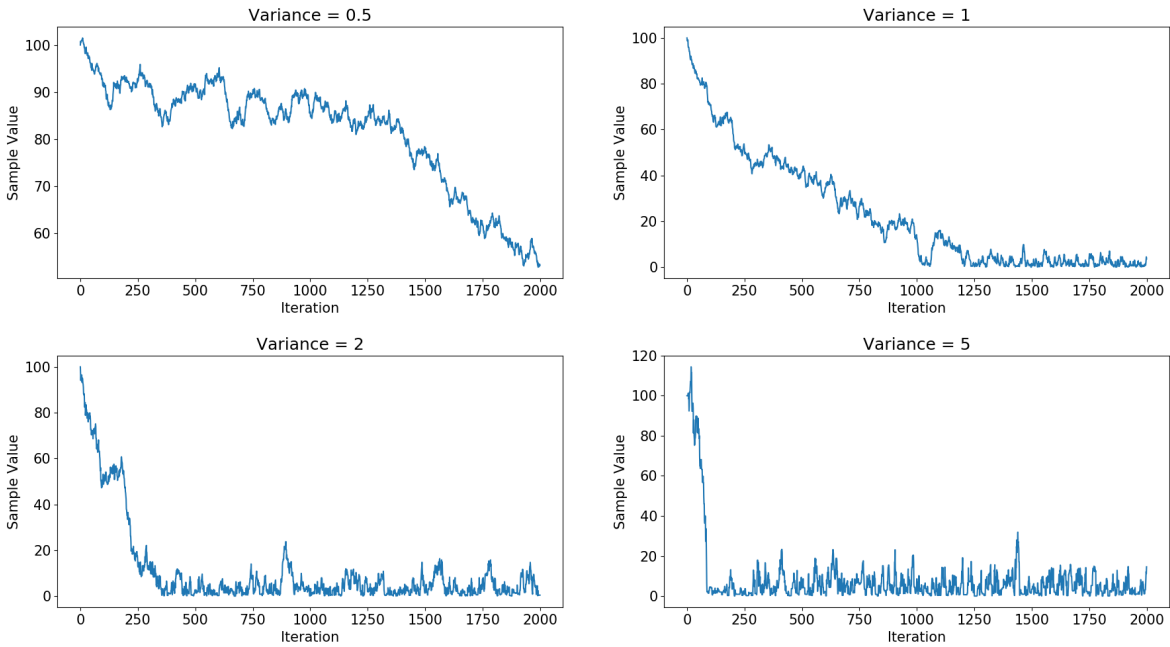


Figure 7: Chain convergence versus variance of normal distribution

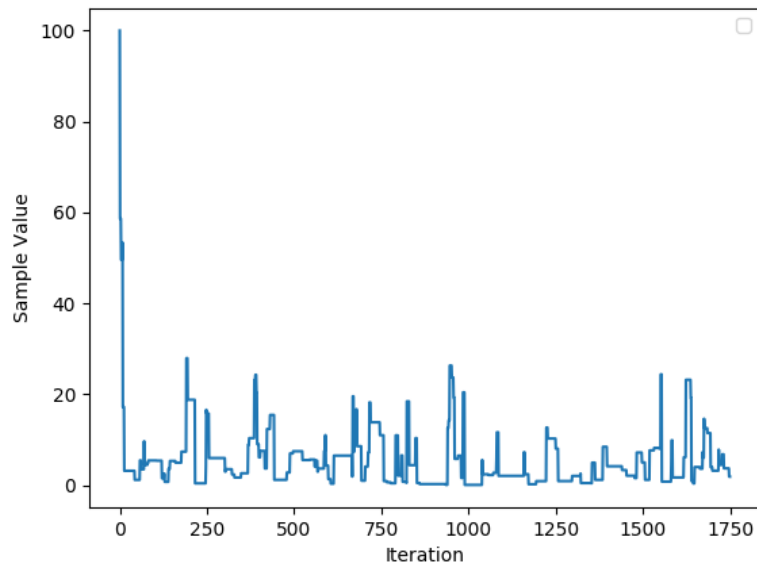
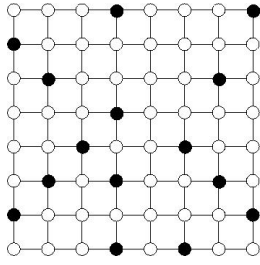


Figure 8: An example showing a variance which is too large

4. (10 points) [HARD-CORE WITH MCMC::GIBBS]



Consider a (non-complete) connected graph $G = (V, E)$ such as the one shown with $n = |V|$. Now each vertex in V gets either mapped to 0 or 1, where we only consider the following set $C \subset \{0, 1\}^n$ of admissible configurations characterized by the property that pairs of adjacent vertices **cannot both** take the value 1 (see figure where black denotes 1).

Now, we want to pick one of the admissible configurations $\mathbf{x} \in C$ "at random". That is, we consider the (discrete) uniform distribution π on C , i.e. $\pi_{\mathbf{x}} = \frac{1}{|C|} \forall \mathbf{x} \in C$.

- (a) (3 points) Write down the edge potentials of the undirected graphical model for this problem, and a Gibbs sampling algorithm for sampling from this model (including how you derived the associated conditional $P(X_i | X_{-i})$). Does your algorithm need to know the partition function $|C|$?

Solution: The edge potentials are as follows:

$$\phi(X_i, X_j) = \begin{cases} 0, & \text{for } X_i = X_j = 1 \\ \lambda, & \text{otherwise} \end{cases}$$

Where λ is any positive constant.

The Gibbs sampling algorithm is as follows:

```

Procedure Gibbs Sample (
    n    // The size of grid to be sampled
    X0  // A random initial state
    T    // Number of time steps
)

1  For t = 1 to T
2      For i = 1 to n2
3          Sample Xit from P(Xi | Nbrs(Xi))
4          Xit ← Xit-1
5  return X0, X1, X2, ..., Xn2

```

The conditionals are derived as follows:

$$P(X_i | X_{-i}) = \frac{P(X_i, X_{-i})}{\sum_{\text{val}(X_i)} P(X_i, X_{-i})}$$

$$= \frac{\frac{1}{Z} \prod_{D_j \ni X_i} \phi(X_i, X_{-i}) \prod_{D_j \not\ni X_i} \phi(X_i, X_{-i})}{\sum_{\text{val}(X_i)} \frac{1}{Z} \prod_{D_j \ni X_i} \phi(X_i, X_{-i}) \prod_{D_j \not\ni X_i} \phi(X_i, X_{-i})}$$

Where, $D_j \ni X_i$ is all the factors containing X_i in their scope.

$$\begin{aligned} &= \frac{\frac{1}{Z} \prod_{D_j \ni X_i} \phi(X_i, X_{-i}) \prod_{D_j \not\ni X_i} \phi(X_{-i})}{\sum_{\text{val}(X_i)} \frac{1}{Z} \prod_{D_j \ni X_i} \phi(X_i, X_{-i}) \prod_{D_j \not\ni X_i} \phi(X_{-i})} \\ &= \frac{\prod_{D_j \ni X_i} \phi(X_i, X_{-i})}{\sum_{\text{val}(X_i)} \prod_{D_j \ni X_i} \phi(X_i, X_{-i})} \end{aligned} \quad (5a)$$

For example, consider a case when one of the neighbours of a node have value 1.

$$\begin{aligned} \text{Then, } P(X_i = 1 | X_{-i}) &= \frac{\prod_{D_j \ni X_i} \phi(X_i = 1, X_{-i})}{\sum_{\text{val}(X_i)} \prod_{D_j \ni X_i} \phi(X_i, X_{-i})} \\ &= \frac{0 \times \lambda \times \lambda \times \lambda}{0 \times \lambda \times \lambda \times \lambda + \lambda \times \lambda \times \lambda \times \lambda} \\ &= 0 \end{aligned}$$

From Equation 5a, the algorithm does not need to know the partition function.

- (b) (3 points) Is the Markov chain you set up irreducible and aperiodic? Does your chain admit a distribution that satisfies detailed balance? If it simplifies your proof, assume here that your Gibbs sampling routine employs “random scan” (pick a random i from $1, \dots, n$ and then make a move based on $P(X_i | X_{-i})$ in each epoch) instead of “systematic scan” (cycle through all i from 1 to n in each epoch).

Solution: When we do Gibbs sampling, we decompose our transition model into multiple transition models and update only one component of the state space at one time. The global chain T has a step which is the aggregate of all local steps.

Aperiodic: The global Markov Chain, which is over the state space $|C|$, is both aperiodic

and irreducible. It is aperiodic because one can stay in the same configuration with a finite probability i.e. there is a self-loop in each state with a non-zero probability. For example, given a permissible state, there is always a finite probability that all the nodes in state 1 stay in state 1 and all others stay in state 0.

Irreducible: It is irreducible since you can reach any state from any other state in a finite number of steps. For example, one can always reach the state where all nodes are zero from any permissible state in a finite number of steps. From this state, any other permissible state can be reached, once again, in a finite number of steps.

Detailed Balance: To prove that there is a distribution $\pi_x(x) = \frac{1}{|C|}$ satisfies detailed balance, we need to prove:

$$\pi_x(x)P(x \rightarrow x') = \pi_{x'}(x')P(x' \rightarrow x)$$

Where x, x' are random states.

$$\pi_x(x)P(x \rightarrow x') = \frac{1}{|C|} \frac{1}{2m}$$

Where m is the number of nodes. Similarly,

$$\pi_{x'}(x')P(x' \rightarrow x) = \frac{1}{|C|} \frac{1}{2m}$$

Thus, detailed balance is satisfied by the distribution $\pi_x(x)$.

- (c) (4 points) Provide your code and trace plots (of some functions that each map a configuration to a real value that helps visualize how well the chain is mixing). Plot the burn-in time you chose as a function of the size of the grid graphs you used. We didn't ask about the empirical acceptance rate here as it is always 100% for Gibbs sampling - prove that it is so under the same "random scan" epoch assumption above.

Solution:

```
import numpy as np
import pandas as pd
from scipy.stats import gamma
import matplotlib.pyplot as plt
```

```

class Gibbs():

    def __init__(self, size, iterations):
        self.size = size
        self.iterations = iterations
        self.grid = np.random.randint(2, size=(size, size))
        print(self.grid)

    def gibbs(self):

        history = np.array(self.grid)

        for i in range(self.iterations):
            for j in range(self.size**2):
                idx = np.unravel_index(j, (self.size, self.size))
                self.grid[idx] = np.random.choice([0,1])
                print(self.grid)
                history = np.dstack((history, self.grid))
                # print(history)
            return(history)

    def conditional(self, idx):

        neighbours = []
        if idx+self.size < self.size**2:
            neighbours.append(idx+self.size)
        if idx-self.size > 0:
            neighbours.append(idx-self.size)
        try:
            if np.unravel_index(idx-1,
                               (self.size, self.size))[0] == np.unravel_index(
                               (self.size, self.size))[0]:
                neighbours.append(idx-1)
        except:
            pass
        try:
            if np.unravel_index(idx+1,
                               (self.size, self.size))[0] == np.unravel_index(
                               (self.size, self.size))[0]:
                neighbours.append(idx+1)

```

```

        except:
            pass
        # print(neighbours)
        return(self.kernel(idx, neighbours))

    def kernel(self, idx, neighbours):

        for idx in neighbours:
            if self.grid[np.unravel_index(idx,
                                           (self.size, self.size))] == 1:
                return([1, 0])
        return([0.5, 0.5])

if __name__ == "__main__":

    G = Gibbs(10, 10)
    history = np.array(G.gibbs())

```

The burn-in time is always one global epoch. After one global epoch, an admissible configuration is always reached. A global epoch takes n^2 smaller steps. Thus, the burn-rate would be of the order of n^2 (in terms of local steps), where n is the size of the grid.

Sanity Check: Figure 9 shows random permissible configurations sampled using the code above.

Trace Plots: Figure 10 shows trace plots of the number of adjacent ones at each local update. After the chain mixes (which happens after one global update), there are no more adjacent ones.

Burn-In Plot: As discussed above, the burn in time will be of the order of n^2 local steps, where n is the size of the grid. This has not been plotted since it is trivial.

Acceptance Rate Proof: Gibbs sampling is a special case of Metropolis Hastings algorithm where, a single dimension is chosen randomly and the proposal distribution is taken as the conditional $P(x^{t+1} | x^t)$.

Under the random scan assumption,

$$\text{Acceptance Probability, } \alpha = \frac{P(x^{t+1})Q(x_t | x^{t+1})}{P(x^t)Q(x^{t+1} | x^t)} \quad (6a)$$

From the above argument,

$$\alpha = \frac{P(x^{t+1})P(x^t | x^{t+1})}{P(x^t)P(x^{t+1} | x^t)} \quad (6b)$$

$$\alpha = \frac{P(x_i^{t+1} | x_{-i}^{t+1})P(x_{-i}^{t+1})P(x_i^t | x_{-i}^{t+1})}{P(x_i^t | x_{-i}^t)P(x_{-i}^t)P(x_i^{t+1} | x_{-i}^t)} \quad (6c)$$

Since all other dimensions remain unchanged from time t to $t + 1$, $P(x_{-i}^t) = P(x_{-i}^{t+1})$,

$$\therefore \alpha = \frac{P(x_i^{t+1} | x_{-i}^{t+1})P(x_{-i}^{t+1})P(x_i^t | x_{-i}^{t+1})}{P(x_i^t | x_{-i}^{t+1})P(x_{-i}^{t+1})P(x_i^{t+1} | x_{-i}^{t+1})} = 1 \quad (6d)$$

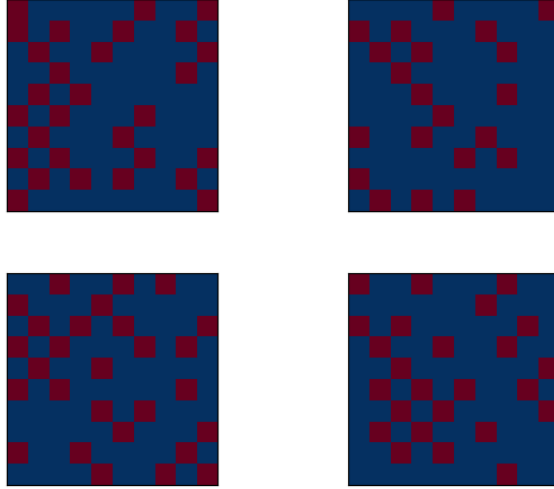


Figure 9: Random permissible configurations sampled using the above algorithm. Blues are zeros and reds are ones.

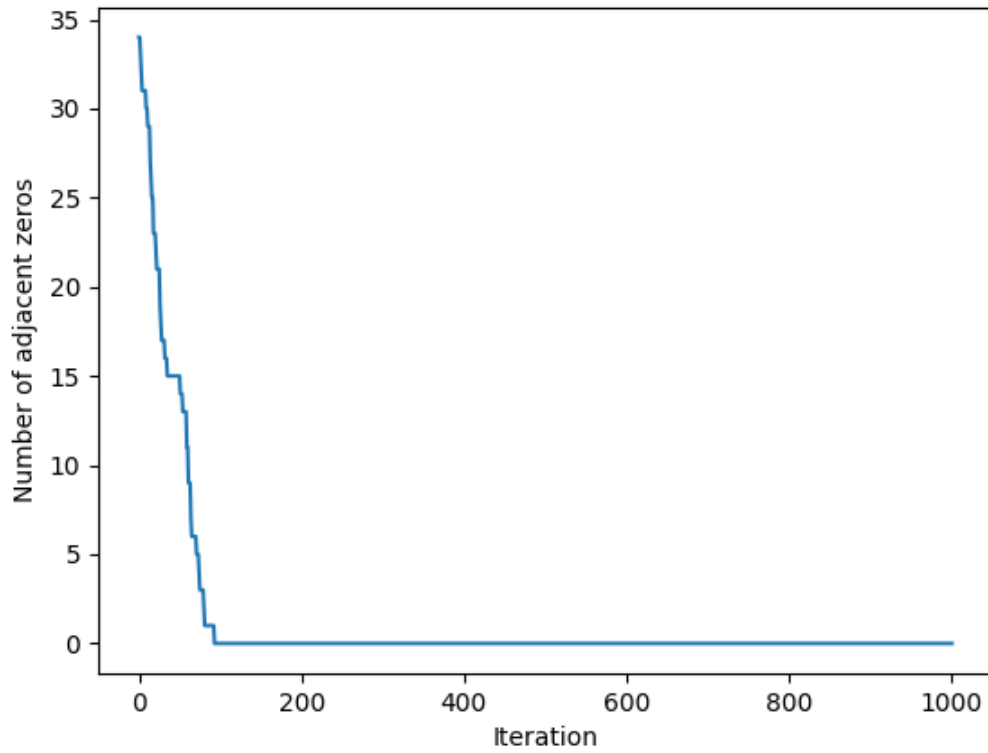


Figure 10: The number of adjacent ones in the grid for each local update. Once the chain mixes, there are no more adjacent ones.